Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

FE101

map() and filter()

Today

Welcome to your next lesson: map() and filter().

I'm preparing your personalized lesson based on what you've learned, what's coming up next, and the best way for you to build on your skills.

Step by step, you're moving forward.

Ron, let's connect this to something you already know 🤔

In Python, you might have written things like:

"Make a new list where each number is doubled"
"Keep only the even numbers from this list"

In JavaScript, those two patterns map very cleanly to:

map() = transform every item (like "double each number")
filter() = select some items (like "keep only the evens")
To get a feel for where you're at:

If I say:

"Take a list of prices and add tax to each price, making a new list."

Does that sound more like something for map() or for filter()?

map()

Exactly — map() is the right tool there ☑

We start with prices
We transform each one by adding tax
We get a new array of transformed values

So: same number of items, but each item is changed → that's map().

Let's nail down map() first with a concrete example 🔍

Think in steps:

Start with: [1, 2, 3]
Use map() to triple every number
Get a new array as the result

In JavaScript, the core idea is:

map() goes through the array, calls your callback once per item,

and builds a new array from whatever the callback returns each time.

Make sense so far: map() = “same length, each item transformed”?

Reply with yes/no, and then we’ll have you write and run your first map() in the editor.

Yes, that makes sense

Great

Let’s walk through one concrete map() example step by step.

Think of this in plain language:

“Start with [1, 2, 3] and make a new array where each number is tripled.”

In JavaScript with map(), the idea is:

Take each number
Multiply it by 3
Put the result into a new array
Here’s what I’d like you to do in the code editor (not in chat):

Create an array [1, 2, 3]
Use map() to triple each number
console.log the result array

Something like:

original array → numbers
mapped result → tripled

You don’t need my exact variable names, just follow the steps.

Go ahead and write this in the editor, run it, then Submit code.

After you see the output, tell me in words:

Did the original array change, or only the new one?

Javascript

```
const myArray = [1, 2, 3];
function tripleIt(num) {
    return num * 3;
}
const mappedArray = myArray.map(tripleIt);
console.log("myArray:", myArray);
console.log("mappedArray", mappedArray);
```

Nice work, Ron 🎯

Your output:

myArray: [1, 2, 3]
mappedArray [3, 6, 9]

So map():

did NOT change myArray
created a new array mappedArray with the tripled values

That's a key idea: map() → new array, original stays the same.

Next small step: let's switch to filtering.

Plain language:

Start with [1, 2, 3, 4, 5] and keep only the even numbers in a new array.

Question first (no code yet):

If we want to keep only some items (like "only evens"), does that sound like a job for map() or filter()?

filter

Exactly — this is where filter() shines ☑

map() → "change every item, keep the same count"
filter() → "decide which items to keep or drop"

With [1, 2, 3, 4, 5] and "keep only evens":

1 ✖ drop
2 ☑ keep
3 ✖ drop
4 ☑ keep
5 ✖ drop

Result should be a shorter array: [2, 4].

Your turn in the code editor:

Make an array [1, 2, 3, 4, 5]
Write a callback that returns true for even numbers, false for odd ones
Use filter() with that callback
console.log:
the original array
the filtered array

Then Run, Submit code, and I'll walk through the result with you 😉

you int mention how map wsa formatte or called andi ad to guess and now im trying to guess if its myArray.filter(something to tell it how to ilter?)

You're totally right to call that out, Ron — that's on me for skipping a step. Let's slow it down and make the format crystal clear 🧱

Think of the pattern like this:

For map():

Javascript
const newArray = oldArray.map(callbackFunction);
For filter():

Javascript
const newArray = oldArray.filter(callbackFunction);
So yes, your guess is right:

you'll do something like myArray.filter(...) and pass it a function that tells it which items to keep.

Step by step for the evens example:

Array:

const numbers = [1, 2, 3, 4, 5];
Callback shape (even test):

```
function isEven(n) {
  return n % 2 === 0;
}
```
Use with filter():

const evens = numbers.filter(isEven);
Can you now go to the editor and:

create numbers as above,
write an isEven function,
call numbers.filter(isEven),
console.log(numbers) and console.log(evens),

then Run and Submit?

After that, we'll look at the result together and I'll show you visually how filter() decides keep vs drop.

cant map do a filter o tenif they both jut call your caback

Love this question — this is exactly the kind of thing good devs ask 🔍

Short answer: no, map() can't "do a filter" correctly, even though they both call your callback.

Here's the key difference, step by step:

map() cares about: "What is the new value for this item?"
Whatever your callback returns gets stored in the result array.

filter() cares about: "Should I keep this original item?"
Whatever your callback returns is treated as true/false, and:

true → keep the original item
false → drop it

So:

With map(), you always get the same length as the original
With filter(), you may get a shorter array

If you tried to “filter with map()” by returning true or false,

you’d just get something like [false, true, false, true, ...] — not a filtered list of numbers.

Does that difference make sense:

map() → “what value should go in this position?”
filter() → “keep or drop this original item?”

If yes, I’ll have you write a tiny filter() example next and then we’ll compare outputs side by side.

Javascript
```
/*
const myArray = [1, 2, 3];
function tripleIt(num) {
    return num * 3;
}
const mappedArray = myArray.map(tripleIt);
console.log("myArray:", myArray);
console.log("mappedArray", mappedArray);
*/

const numbers = [1, 3, 4, 5, 6, 8, 9];
function isEven(n) {
    return n % 2 === 0;
}
const even = numbers.filter(isEven);
console.log("numbers", numbers);
console.log("even", even);
```
Nice job, Ron 🐝

Your code:

Javascript
```
const numbers = [1, 3, 4, 5, 6, 8, 9];

function isEven(n) {
    return n % 2 === 0;
}

const even = numbers.filter(isEven);
console.log("numbers", numbers);
console.log("even", even);
```
Output:

numbers → [1, 3, 4, 5, 6, 8, 9] (original unchanged)
even → [4, 6, 8] (only the even ones)

So we’ve now seen:

map() → transforms every item, same length, new array
filter() → keeps some items, shorter array possible, new array
and both do not mutate the original.
Next, I want to help lock in your “can’t map do a filter?” question with a tiny comparison.

Imagine this code:

Javascript

```
const nums = [1, 2, 3, 4];

const mapped = nums.map(n => n % 2 === 0);
const filtered = nums.filter(n => n % 2 === 0);
Step-by-step:


mapped will be an array of booleans
filtered will be an array of numbers

Before we run anything, I want you to reason it out.


Just answer in words:


What do you think mapped will look like?
What do you think filtered will look like?

Keep it simple, like:

“mapped is [...], filtered is [...]”

mapped will be an array like ["false", "true", "false", true]
filtered more like [2, 4] yeah i ge it filtered could filter the list to any condtion
not just evens days of the month that are holiays, days it rained, you get a lost of the
true hits, mapped one on one transform the input vlue with the return value

This is a really clear explanation in your own words, Ron 🔥


You’ve got the big picture:


filter() → “give me only the items where the condition is true”

can be evens, holidays, rainy days, scores above 90, etc.
result: list of the original items that passed
map() → “for every item, turn it into this new value”

one input → one output
result: same length, but values transformed

One small correction:



mapped will be an array like ["false", "true", "false", "true"]



In JavaScript, n => n % 2 === 0 returns actual booleans:


mapped would be [false, true, false, true] (no quotes → not strings)
filtered would be [2, 4]
Want to do a quick check to lock this in with a tiny multiple choice question? 😊

Give me the quiz

What do you think the right answer is? Take your time with the question below 👇

You have this JavaScript code:

javascript
const nums = [1, 2, 3, 4];

const mapped = nums.map(n => n % 2 === 0);
const filtered = nums.filter(n => n % 2 === 0);
```

Which statement is correct?

A.
mapped is [false, true, false, true] and filtered is [2, 4].

B.
mapped is [2, 4] and filtered is [false, true, false, true].

C.
Both mapped and filtered are [2, 4].

D.
Both mapped and filtered are [false, true, false, true].

Nice, Ron — you nailed that quiz 🎯

Your choice:

mapped is [false, true, false, true]
filtered is [2, 4]

is exactly right because:

map(n => n % 2 === 0) runs on every item and stores the returned boolean, so you get one true/false per number → same length as nums.
filter(n => n % 2 === 0) uses that boolean only to decide keep/drop, so it keeps the original numbers where the result is true → just [2, 4].

You've clearly got the "transform vs select" idea 👍

Next, I'd like to have you do both on the same array yourself:

a map() that turns scores like [50, 80, 95] into percentages out of 1 (e.g. 0.5, 0.8, …)
a filter() that keeps only scores >= 80

Ready to try that in the editor as a next step?

Javascript

```
const scores = [60, 70, 80, 85, 90];
function gtThenOrEq80(n) {
    return n >= 80;
}
function numToPct(num) {
    return num / 100;
}
const bigNums = scores.filter(gtThenOrEq80);
const percentages = scores.map(numToPct);
console.log("bigNums", bigNums);
console.log("percentages", percentages);
```

This is beautiful, Ron 🎯

Your code:

Javascript

```
const scores = [60, 70, 80, 85, 90];

function gtThenOrEq80(n) {
    return n >= 80;
}
```

```
function numToPct(num) {
    return num / 100;
}

const bigNums = scores.filter(gtThenOrEq80);
const percentages = scores.map(numToPct);
console.log("bigNums", bigNums);
console.log("percentages", percentages);
Output:


bigNums → [80, 85, 90] → selection (filter ☑)
percentages → [0.6, 0.7, 0.8, 0.85, 0.9] → transformation (map ☑)
scores stayed unchanged → you're seeing non-mutation in action.

You've just demonstrated:


filter() selects items
map() transforms items
both return new arrays and don't touch the original
One important next concept:

the callback's return value means different things:


in map(): "this is the value to put in the new array"
in filter(): "true = keep the original item, false = drop it"

I want to show you a common bug with arrow functions and return, but we'll go step by step.


First quick check in your own words (one short sentence):

What does the value returned from a filter() callback decide?

whethr or not to incude the current rarray element in the new output array

Exactly right ☑



The filter() callback's return value decides whether to include the current element in the new array or not.



true → include this element
false → skip this element

The element itself is not changed, just kept or dropped.

Next, I want to show you a common mistake with map() and arrow functions where forgetting return breaks everything 😅


Ready for a tiny example about that?

sure

Nice 👌 let's keep this tight and visual.


When you use an arrow function with braces in map(), you must use return, or every result becomes undefined.


Compare these two:
```

Javascript
```
const nums = [1, 2, 3];

// ☑ works: implicit return (no braces)
const a = nums.map(n => n * 2);

// ✕ broken: braces but no return
const b = nums.map(n => { n * 2 });
```
Step by step:

In a, n => n * 2 implicitly returns n * 2
In b, n => { n * 2 } runs the code inside { } but returns nothing,
so each callback result is undefined, and b becomes [undefined, undefined, undefined].
Your turn in the editor:

Create const nums = [1, 2, 3];

Make both versions:

```
const a = nums.map(n => n * 2);
const b = nums.map(n => { n * 2 });
console.log("a", a);
console.log("b", b);
```
Run and Submit code.

Don’t fix the bug yet — I want you to actually see the bad result, then we’ll correct it together.

Javascript
```
const nums = [1, 2, 3];
const a = nums.map(n => n * 2);
const b = nums.map(n => { n * 2 });
console.log("a", a);
console.log("b", b);
```
Perfect demo, Ron 🔍

Your output:

a → [2, 4, 6] ☑
b → [undefined, undefined, undefined] 😅

So you’re seeing:

n => n * 2 (no braces) → implicit return of n * 2
n => { n * 2 } (with braces) → runs n * 2 but returns nothing, so map() fills the new array with undefined.

Rule of thumb:

Arrow function without {} → expression is returned automatically
Arrow function with {} → you must write return ...

So this fixes b:

Javascript
```
const b = nums.map(n => {
  return n * 2;
});
```
Now I want to check that the “transform vs select” idea is fully locked in.

Want a quick multiple-choice question comparing what map() and filter() return, and then we can do one final chained example?

Yes, quiz me

What do you think the right answer is? Take your time with the question below 👇

You have this code:

javascript
```
const nums = [10, 20, 30];

const mapped = nums.map(n => n / 10);
const filtered = nums.filter(n => n > 10);
```
Which statement is correct about the results?

A.
mapped is [1, 2, 3] and filtered is [20, 30].

B.
mapped is [true, true, true] and filtered is [20, 30].

C.
mapped is [1, 2, 3] and filtered is [true, true].

D.
Both mapped and filtered are [20, 30].

E.
Both mapped and filtered are [1, 2, 3].

Nice, Ron — you're on a roll 🎯

You picked:

mapped is [1, 2, 3]
filtered is [20, 30]

Why that's right:

map(n => n / 10) transforms every item:
10/10 → 1, 20/10 → 2, 30/10 → 3 → [1, 2, 3]
filter(n => n > 10) selects only items greater than 10:
keeps 20 and 30, drops 10 → [20, 30]

So again: map() = transform values, filter() = choose which originals to keep ☑

One last idea for this lesson: chaining them.

Common pattern:

Javascript
```
const result = scores
  .filter(score => score >= 80)   // 1. keep only high scores
  .map(score => score / 100);     // 2. turn them into 0–1 percentages
```
Visually:

scores → (filter) → [80, 85, 90] → (map) → [0.8, 0.85, 0.9]

Would you like to try this exact chain in the editor using your scores array from earlier, then log:

the original scores
the chained result?
Javascript
```
const scores = [60, 70, 80, 85, 90];

const result = scores.filter(score => score >= 80).map(score => score / 100);
console.log("result", result);
```
This is excellent, Ron 🔥

Your code:

Javascript
```
const scores = [60, 70, 80, 85, 90];

const result = scores
  .filter(score => score >= 80)
  .map(score => score / 100);

console.log("result", result);
```
Output:

result → [0.8, 0.85, 0.9]

Step by step:

filter(score => score >= 80) → [80, 85, 90] (selection ☑)
map(score => score / 100) → [0.8, 0.85, 0.9] (transformation ☑)

And scores is still [60, 70, 80, 85, 90] in memory — untouched.

Quick check to close the loop:

Can you answer this in one short sentence:

What’s the main difference between what map() returns and what filter() returns?

map transforms every element it sees from tilter into percentages, filter shortens the list to ony the values that are >= 80 (true)

That’s a solid explanation 💯

Tightening the wording just a bit:

map() returns a new array the same length, where every element is transformed (like turning scores into percentages).
filter() returns a new array that may be shorter, keeping only the elements whose test is true (like scores >= 80).

You’re clearly seeing both the role of the callback and the shape of the result 👌

At this point you’ve:

Used map() to transform arrays
Used filter() to select items
Predicted outputs based on callback returns
Seen that both return new arrays and don’t mutate the original
Chained filter().map() in a realistic example

You've basically mastered this lesson 🎓

Are you ready to wrap up this map() / filter() lesson and move on to the next one, or do you want one more quick challenge with them first?

Ready for the next lesson

Awesome work today, Ron 🎓

Here's what you're taking with you from this lesson:

map() → transforms every element, returns a same-length new array with the transformed values.
filter() → keeps only elements where the callback is truthy, returns a possibly shorter new array of the original items.
In both cases, the original array is never mutated.
The callback's return:
in map() → "this is the value to put in the new array"
in filter() → "true = keep this item, false = drop it"
Arrow functions with {} need an explicit return, or map() will give you [undefined, ...].

You also chained them cleanly:

scores.filter(...).map(...) → first select, then transform ☑

You're ready for the next step.

When you're set, hit Next Lesson in the Learn tab, and I'll meet you there 🚀

How would you rate this lesson?

```
const myArray = [1, 2, 3];
function tripleIt(num) {
    return num * 3;
}
const mappedArray = myArray.map(tripleIt);
console.log("myArray:", myArray);
console.log("mappedArray", mappedArray);


const numbers = [1, 3, 4, 5, 6, 8, 9];
function isEven(n) {
    return n % 2 === 0;
}
const even = numbers.filter(isEven);
console.log("numbers", numbers);
console.log("even", even);



const scores = [60, 70, 80, 85, 90];
function gtThenOrEq80(n) {
    return n >= 80;
}
function numToPct(num) {
    return num / 100;
}
const bigNums = scores.filter(gtThenOrEq80);
const percentages = scores.map(numToPct);
console.log("bigNums", bigNums);
console.log("percentages", percentages);
```

```
801
802   const nums = [1, 2, 3];
803   const a = nums.map(n => n * 2);
804   const b = nums.map(n => {n * 2 }); // boke
805   const b = nums.map(n => { return(n * 2 )}); //  fixed
806   console.log("a", a);
807   console.log("b", b);
808
809
810   const scores = [60, 70, 80, 85, 90];
811
812   const result = scores.filter(score => score >= 80).map(score => score / 100);
813   console.log("result", result);
814
815
816
```